



INSTITUTO SUPERIOR TÉCNICO

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

ORGANIZAÇÃO DE COMPUTADORES

LEIC

Second Lab Assignment: TLB Simulator

STUDENTS IDENTIFICATION:

Number:	Name:
57175	João Pedro dos Santos Carvalho
105882	Duarte Mendonça Gouveia
107032	Rodrigo Alexandre Almeida Fonseca dos Santos

1. Introdução

Este relatório descreve a implementação de um sistema de TLB (Translation Lookaside Buffer) de dois níveis, L1 e L2, em C. A TLB é uma cache usada para acelerar a tradução de endereços virtuais para endereços físicos. Para aumentar a eficiência, o código implementado mantém cópias de traduções e usa LRU (Least Recently Used) e Write-Back para otimizar o desempenho.

2. Estrutura de Dados

Cada entrada da TLB contém os seguintes argumentos:

- `valid`: Indica se a página tem dados válidos
- `dirty`: Marca se a página foi modificada
- `last_access`: timestamp do último acesso
- `virtual_page_number`: número da página virtual
- `physical_page_number`: número da página física correspondente

Este sistema implementa também 2 arrays:

- `tlb_L1[TLB_L1_SIZE]` : cache L1 (a mais rápida mas com uma capacidade pequena)
- `tlb_L2[TLB_L2_SIZE]` : cache L2 (mais lenta que L1 mas com uma capacidade muito superior)

3. Principais Funcionalidades

Após a inicialização de todas as variáveis e arrays, a função `tlb_init()` inicializa ambas as TLBs e coloca todos os parâmetros a 0 para garantir que todas as entradas começam como inválidas.

De seguida, de acordo com a nossa implementação, usamos duas funções auxiliares para realizarem o processo de busca e substituição. Começando com a `find_entry()` que realiza uma procura linear nas entradas válidas onde procura um Virtual Page Number específico. Após esta pesquisa retorna o índice de entrada se esta for encontrada, caso contrário retorna -1.

Posteriormente a função `find_lru_entry()` localiza a entrada menos usada recentemente (LRU). Esta função prioriza entradas inválidas e caso todas sejam válidas retorna a com um menor timestamp.

Por último, a função `tlb_invalidate()` remove uma tradução específica de ambos os níveis da TLB. Começa por procurar o Virtual Page Number em L1 e se este for encontrado executa write-back se a entrada estiver dada como dirty, marca a entrada como inválida, incrementa o contador das invalidações e incrementa o tempo de latência de L1. Repete exatamente o mesmo processo para L2. Esta função é extremamente importante e crítica para manter a coerência quando existem modificações na page table.

4. Tradução de Endereços

A função `tlb_translate()` é a função principal deste programa e que trata de todo o processo de tradução usando hierarquia de caches. Esta começa por dividir o endereço virtual recebido, retirando do mesmo a Virtual Page Number que identifica a página e o Offset que é a posição dentro da página.

Após este processo, tenta encontrar a tradução na TLB L1 que pode resultar em dois cenários. Um deles é encontrar a tradução e por isso resulta num L1 HIT ou pode não encontrar e resulta num L1 MISS. Independentemente de acertar ou falhar custa igualmente 1 ns. Atualiza o tempo de acesso e o bit dirty se se tratar de uma escrita. Retorna imediatamente o endereço físico se a tradução for encontrada.

Caso não seja encontrada, tenta na L2. Se for este o caso, incrementa o L2 HIT e o tempo, se for uma escrita marca a entrada como dirty e obtém o número da página física.

De seguida promove a entrada para a L1, procurando a entrada LRU da L1 para a substituir. Se a entrada da L1 a substituir ainda for válida, verifica se já existe no L2. Se existir propaga o bit dirty, se for necessário. Se não existir move a entrada expulsa de L1 para o L2 e por fim insere a nova tradução que veio da L2 na L1. Por fim retorna o endereço físico correspondente. Caso também falhe na L2 traduz o endereço usando a page table que é muito mais lento, atualiza o número de misses e obtém o Physical Page Number correspondente. Após este processo insere a tradução em L2 e de seguida em L1, retornando o endereço físico final.

Cada vez que uma entrada é dirty, é chamada a função `write_back_tlb_entry()` para sincronizar a página modificada com a memória de maneira a garantir consistência dos dados.

5. Conclusão

A implementação submetida oferece um sistema de TLB hierárquico eficiente com ambas operações de leitura e escrita, política de substituição LRU e gestão adequada de consistência de dados usando o write-back. Após verificar todos os testes disponibilizados e passar todos conseguimos concluir que esta implementação é adequada para sistemas que exigem tradução rápida de endereços virtuais com uma boa taxa de HIT.